

Capítulo

1

ScienceReview: modelagem e implantação de um banco de dados para gestão de submissões e avaliação científica

Filipe dos Santos Freitas, Larissa Vitória Maturino Assis, Leticia Silva Oliveira, Lucas dos Santos Lima e Lucy Cassia Cerqueira Silva

Instituto de Computação – Universidade Federal da Bahia (UFBA)

Av. Milton Santos, s/n – Campus de Ondina – Salvador – BA, 40170-110

Abstract

This chapter presents the conceptual modeling, logical modeling and PostgreSQL deployment of a subset of the database for ScienceReview, an information system for managing scientific submissions and peer review. The scope covers the core review cycle: authorship, scientific topics, reviewer expertise, review assignment, reviews and final decisions. Each of the 45 original requirements is classified as met, partially met or out of scope, four new requirements are proposed as an extension, and a traceability table links requirements to entities and relationships. The database is populated with synthetic data, with three tables exceeding five thousand records, and its catalog is explored through metadata queries.

Resumo

Este capítulo apresenta a modelagem conceitual, a modelagem lógica e a implantação em PostgreSQL do banco de dados do ScienceReview, um sistema de informação para gestão de submissões e avaliação científica. O recorte cobre o núcleo do ciclo de avaliação: autoria, tópicos científicos, expertise de revisores, atribuição de revisões, revisões e decisões finais. Cada um dos 45 requisitos originais é classificado como atendido, parcialmente atendido ou fora do escopo, quatro requisitos novos são propostos como extensão, e uma tabela de rastreabilidade relaciona requisitos, entidades e relacionamentos. O banco é populado com dados sintéticos, com três tabelas acima de cinco mil registros, e seu catálogo é explorado por meio de consultas de metadados.

1.1. Introdução

A Sociedade Brasileira de Tecnologia e Computação (SBTC) promove anualmente dezenas de eventos científicos, alguns com milhares de submissões e centenas de pesquisadores, em que uma mesma pessoa pode ser autora, revisora ou coordenadora em eventos diferentes. Para apoiar esse processo, a SBTC decidiu desenvolver o ScienceReview, um sistema de informação que acompanha cada trabalho do registro à decisão final, em eventos organizados em edições e trilhas com regras próprias [Universidade Federal da Bahia 2026].

Dos 45 requisitos funcionais do enunciado, o grupo delimitou o minimundo ao **ciclo central de avaliação**: uma pessoa submete um trabalho a uma trilha, a coordenação o atribui a revisores com expertise nos tópicos da submissão, os revisores emitem pareceres e a coordenação registra a decisão. Ficam fora do recorte, com justificativa na Seção 1.3, as funcionalidades que dependem de histórico ou de configuração por trilha, como versionamento de manuscritos, formulários configuráveis, réplica, discussão do comitê e auditoria. O modelo resultante atende, total ou parcialmente, 23 dos 45 requisitos originais e os quatro requisitos da extensão proposta.

A Seção 1.2 descreve o problema, os objetivos e a extensão, e a Seção 1.3 classifica os requisitos. As Seções 1.4 a 1.6 apresentam o minimundo, os modelos conceitual e lógico e a rastreabilidade entre modelo e requisitos. As Seções 1.7 e 1.8 tratam da implantação em PostgreSQL [The PostgreSQL Global Development Group 2026], da população e dos metadados, e os anexos trazem os scripts e o roteiro de reprodução.

1.2. Descritivo do Projeto

1.2.1. Problema

Os eventos promovidos pela SBTC recebem trabalhos que precisam ser submetidos, distribuídos entre especialistas, avaliados e selecionados antes da publicação. Nos maiores eventos, com milhares de submissões e centenas de pesquisadores, a coordenação precisa distribuir cada trabalho entre revisores com conhecimento nos seus tópicos, sem violar conflitos de interesse, e acompanhar se cada convite foi aceito e se o parecer foi entregue. A decisão final considera as avaliações, as recomendações dos revisores e outros critérios científicos, e não deve ser reduzida a uma média de notas [Universidade Federal da Bahia 2026].

O problema central deste projeto é, portanto, a alocação e o acompanhamento da avaliação por pares (*peer review*). Sem uma base que relacione pessoas, submissões, tópicos, atribuições, revisões e decisões, a coordenação não consegue cruzar a temática dos trabalhos com a expertise dos revisores, medir a carga de cada revisor nem comparar os pareceres que embasam a decisão.

1.2.2. Objetivos

O objetivo geral deste projeto é modelar e implantar em PostgreSQL um banco de dados relacional para o núcleo do processo de revisão por pares do ScienceReview, da submissão

à decisão final. Os objetivos específicos são:

- Manter um cadastro único de pessoas, no qual os papéis de autor e revisor são definidos pela submissão, e não por uma propriedade fixa da pessoa;
- Relacionar os tópicos das submissões à expertise declarada pelos revisores, para apoiar a escolha de revisores;
- Registrar o ciclo de atribuição (convite, aceite, recusa ou remoção), permitindo medir a carga de cada revisor e sinalizar conflitos de interesse;
- Consolidar os pareceres em critérios fixos de avaliação, permitindo extrair indicadores gerenciais que apoiem a decisão da coordenação, sem substituí-la.

1.2.3. Extensão proposta

A extensão proposta reforça o papel do ScienceReview como apoio analítico à coordenação. Em vez de ampliar a logística de eventos, o grupo acrescenta quatro requisitos que exploram os dados já presentes no recorte para apoiar a atribuição e o acompanhamento das revisões:

RF46 – Aderência temática: O sistema deve calcular, para cada par revisor–submissão, a aderência entre os tópicos da submissão e os tópicos em que o revisor declarou expertise, ponderada pelo nível de expertise, para ordenar os candidatos a revisor de cada submissão.

RF47 – Detecção de conflito por autoria e instituição: O sistema deve sinalizar como conflitosa a atribuição em que o revisor é autor da submissão ou está vinculado à mesma instituição de algum de seus autores.

RF48 – Tempo de resposta dos revisores: O sistema deve permitir medir o intervalo entre a atribuição e a entrega do parecer, por revisor e por evento.

RF49 – Desempenho por instituição: O sistema deve produzir indicadores de número de submissões e de taxa de aceitação por instituição dos autores.

1.3. Requisitos

Tabela 1.1. Situação de cada requisito no modelo e justificativa.

RF	Requisito	Situação	Justificativa e elementos do modelo
RF1	Eventos e edições	Parcial	tbl_evento guarda o evento e ano_edicao; a edição não é entidade própria, e edições de anos diferentes são linhas distintas (Seção 1.4.2).
RF2	Trilhas	Parcial	A trilha é uma linha de tbl_evento ligada ao evento principal (Possui_Trilha), com período de submissão e coordenador próprios; regras, comitê, tópicos e formulários por trilha ficam fora do recorte.

RF	Requisito	Situação	Justificativa e elementos do modelo
RF3	Usuários	Atendido	tbl_pessoa é única na plataforma, com email_principal único; papéis vêm dos relacionamentos.
RF4	Papéis contextuais	Parcial	Autor (É_Autor) e revisor (É_Revisor) dependem da submissão, e o coordenador (Coordena) depende do evento ou da trilha; o coordenador de programa e o comitê ficam fora do recorte.
RF5	Instituições	Parcial	Pertence associa cada pessoa a no máximo uma instituição atual; o histórico de vínculos fica fora do recorte.
RF6	Submissões	Atendido	tbl_submissao tem título, resumo, idioma, data de registro e status; pertence a uma única trilha (Recebe, 1:N).
RF7	Autoria	Atendido	É_Autor exige ao menos um autor por submissão e guarda ordem_autoria e ind_responsavel.
RF8	Alteração de autoria	Fora do escopo	Exige histórico e auditoria de alterações, que não fazem parte do ciclo central.
RF9	Tópicos	Parcial	Aborda relaciona submissões e tópicos (N:N); o tópico não é vinculado à trilha que o define.
RF10	Perfil de expertise	Atendido	Tem_Expertise relaciona pessoa e tópico com nivel_expertise, em escala de 1 a 5.
RF11	Versões	Fora do escopo	Cada submissão guarda um único arquivo (nm_arquivo_pdf); ver Seção 1.4.2.
RF12	Integridade dos arquivos	Fora do escopo	Depende do versionamento (RF11).
RF13	Processo de revisão	Parcial	Uma submissão recebe várias atribuições e revisões; a quantidade desejada de revisores por trilha não é registrada.
RF14	Comitê de Programa	Fora do escopo	O recorte trata revisor como papel em É_Revisor, sem modelar o comitê.
RF15	Atribuição de revisores	Parcial	É_Revisor guarda data (dt_atribuicao) e situação (status_aceite); a origem da atribuição não é registrada.
RF16	Estratégia de atribuição	Fora do escopo	Depende da origem da atribuição, não registrada no recorte.
RF17	Carga de revisão	Parcial	A carga é obtida por contagem em É_Revisor; limites de carga não são registrados.
RF18	Conflitos de interesse	Parcial	ind_conflito sinaliza o conflito na atribuição; o bloqueio prévio à atribuição fica fora do recorte (Seção 1.4.2).
RF19	Origem do conflito	Fora do escopo	O recorte registra apenas a existência do conflito.
RF20	Temporalidade do conflito	Fora do escopo	Depende de histórico de vínculos (RF5).
RF21	Formulários de revisão	Fora do escopo	O recorte usa um conjunto fixo de critérios; ver Seção 1.4.2.
RF22	Questões de avaliação	Fora do escopo	Idem RF21.

RF	Requisito	Situação	Justificativa e elementos do modelo
RF23	Tipos de resposta	Fora do escopo	Idem RF21; os critérios fixos cobrem apenas notas numéricas e texto livre.
RF24	Versão do formulário	Fora do escopo	Depende de formulários configuráveis (RF21).
RF25	Revisões	Parcial	Gera_Parecer liga cada revisão a exatamente uma atribuição e guarda dt_revisao; status da revisão e respostas a formulário ficam fora.
RF26	Recomendação	Atendido	recomendacao_final em tbl_imp_revisao.
RF27	Confidencialidade	Fora do escopo	O parecer é um texto único, sem separação entre comentários aos autores e à coordenação.
RF28	Rebuttal	Fora do escopo	Etapla opcional por trilha, fora do ciclo central.
RF29	Discussão	Fora do escopo	Etapla interna do comitê, fora do ciclo central.
RF30	Rodadas de avaliação	Parcial	Uma atribuição pode gerar várias revisões (Gera_Parecer, 1:N), preservadas e ordenadas por dt_revisao; a rodada não é identificada explicitamente.
RF31	Decisão	Atendido	tbl_imp_decisao guarda resultado e data; o responsável vem de Emite.
RF32	Histórico de decisão	Atendido	Recebe_Veredito é 1:N: decisões anteriores são preservadas, cada uma com resultado, data e responsável (Emite); a decisão vigente é a mais recente.
RF33	Modalidade diferente	Fora do escopo	Depende de modalidades por trilha, não modeladas.
RF34	Notificação	Fora do escopo	Comunicações ficam a cargo da aplicação.
RF35	Versão final	Fora do escopo	Depende do versionamento (RF11).
RF36	Documentos adicionais	Fora do escopo	Depende do versionamento (RF11).
RF37	Prazos	Parcial	Períodos de realização (dt_inicio, dt_fim) e de submissão (dt_inicio_submissao, dt_fim_submissao) por evento ou trilha; prazos de revisão, réplica, decisão e versão final ficam fora.
RF38	Exceções de prazo	Fora do escopo	Depende dos demais prazos (RF37).
RF39	Anonimização	Fora do escopo	Política de anonimato por trilha não é modelada.
RF40	Controle de acesso	Parcial	Os papéis por submissão permitem decidir o acesso; fase do processo e coordenação ficam fora.
RF41	Auditoria	Fora do escopo	Exige registro de operações, fora do ciclo central.
RF42	Indicadores gerenciais	Parcial	Suportados: i, ii, iii, vii, viii (conflitos sinalizados), ix, x, xi, xii, xiii e xv. Não suportados: iv, v, vi e xiv.

RF	Requisito	Situação	Justificativa e elementos do modelo
RF43	Reprodutibilidade	Fora do escopo	Exige histórico de estados e de versões; apenas revisões e decisões são preservadas.
RF44	Integridade	Parcial	Chaves primárias, unicidade (e-mail, ORCID, sigla por edição, revisor por submissão e ordem de autoria), integridade referencial e domínios de status, notas e datas validados por CHECK no DDL; regras que envolvem mais de uma linha ou tabela (ex.: ao menos um autor responsável) ficam a cargo da aplicação.
RF45	Escalabilidade	Parcial	Índices nas chaves estrangeiras usadas nas junções (Seção 1.7); o volume de teste (Seção 1.8) é menor que o de produção, e a análise de desempenho fica para o Marco 2.
Requisitos da extensão (Seção 1.2.3)			
RF46	Aderência temática	Atendido	Aborda e Tem_Expertise ligam submissão e revisor aos mesmos tópicos; nivel_expertise (1 a 5) pondera a aderência.
RF47	Conflito por autoria e instituição	Atendido	É_Autor e Pertence permitem detectar o conflito, registrado em ind_conflito; considera apenas a instituição atual (RF5).
RF48	Tempo de resposta	Atendido	Diferença entre dt_atribuicao (É_Revisor) e dt_revisao (tbl_imp_revisao), ligadas por Gera_Parecer.
RF49	Desempenho por instituição	Atendido	Pertence, É_Autor e Recebe_Veredito ligam instituição, submissão e decisão; considera apenas a instituição atual (RF5).

1.4. Minimundo

1.4.1. Descrição do minimundo

O minimundo restringe o ScienceReview ao ciclo central de submissão e avaliação por pares. O modelo conceitual tem sete entidades e onze relacionamentos.

O sistema mantém um cadastro único de **pessoas**. Cada pessoa tem nome, e-mail principal único, país e, opcionalmente, um identificador ORCID, também único, e pode estar vinculada a uma **instituição**, identificada por nome, sigla, país, cidade e UF.

Cada **evento** tem sigla, nome, descrição, ano da edição, período de realização, período de submissão e uma pessoa coordenadora. Uma trilha é registrada como um evento filho, que referencia o evento principal e tem período de submissão e coordenador próprios.

Cada **submissão** pertence a uma única trilha e tem título, resumo, idioma, nome do arquivo do manuscrito, data de registro e status. Uma submissão tem um ou mais autores, cada um com uma posição na ordem de autoria, e ao menos um deles é responsável pela

submissão. Uma pessoa pode ser autora de várias submissões. Cada submissão aborda um ou mais **tópicos**, e uma pessoa pode declarar expertise em vários tópicos, com um nível para cada um.

A coordenação atribui submissões a pessoas que atuarão como revisoras. Cada atribuição registra a data, a situação do convite (pendente, aceito, recusado ou removido) e se há conflito de interesses sinalizado. Toda **revisão** nasce de exatamente uma atribuição, e uma atribuição pode gerar mais de uma revisão ao longo do processo. A revisão registra data, notas de originalidade, metodologia e relevância, parecer textual e recomendação final. Por fim, pessoas da coordenação emitem **decisões** sobre a submissão, cada uma com resultado, data e justificativa opcional; uma nova decisão não apaga as anteriores.

Ficam fora do minimundo o versionamento de arquivos, os comitês de programa, os formulários configuráveis, a anonimização, as notificações, a réplica (*rebuttal*), a discussão interna e a auditoria.

1.4.2. Decisões sobre o texto base

O grupo tomou as seguintes decisões sobre a especificação original, todas motivadas pelo recorte no ciclo central de avaliação:

1. **Evento e trilha na mesma entidade:** O texto base propõe `tbl_evento`, `tbl_edicao` e `tbl_trilha`. Como trilhas e eventos compartilham os mesmos atributos (sigla, nome, períodos e coordenador), a trilha foi modelada como um evento filho, por meio de um autorrelacionamento em `tbl_evento` (`Possui_Trilha`). A edição foi reduzida ao atributo `ano_edicao`, porque nenhum requisito do recorte depende de dados próprios da edição. O custo assumido é que o banco não impede uma trilha de ter suas próprias trilhas, nem uma submissão de apontar para um evento principal; essas regras ficam a cargo da aplicação e serão revistas na normalização do Marco 2.
2. **Coordenador por evento ou trilha:** Cada evento e cada trilha tem uma pessoa coordenadora (`Coordena`), o que atende parcialmente aos papéis contextuais (RF4). O coordenador de programa e o comitê ficam fora do recorte.
3. **Critérios de revisão como atributos fixos:** O módulo de formulários configuráveis, que exigiria um modelo do tipo *Entity-Attribute-Value*, foi substituído por três critérios citados no texto base (originalidade, metodologia e relevância), materializados como colunas numéricas em `tbl_imp_revisao`. A decisão abdica da flexibilidade por trilha (RF21–RF24) em favor de consultas de agregação diretas (AVG, STDDEV), que apoiam a comparação de pareceres sem substituir a decisão da coordenação.
4. **Separação entre atribuição e parecer:** O convite ao revisor (`É_Revisor`) e a entrega do parecer (`tbl_imp_revisao`) foram modelados separadamente. Assim, atribuições pendentes ou recusadas não geram linhas de revisão com colunas vazias (`NULL`).

5. **Arquivo único por submissão:** A `tbl_versao_submissao` do texto base foi substituída pelo atributo `nm_arquivo_pdf` em `tbl_submissao`. O ciclo central depende apenas do manuscrito vigente, e não do histórico de envios; por isso, RF11, RF12, RF35 e RF36 ficam fora do escopo.
6. **Conflito sinalizado na atribuição:** O conflito de interesses é registrado no atributo `ind_conflito` de `É_Revisor`. No recorte, o conflito é avaliado no momento da atribuição; o registro prévio, a origem e a temporalidade do conflito (RF19 e RF20) ficam fora do escopo.
7. **Prefixo de implementação:** Conforme o item 2.1 do enunciado, as entidades de revisão e de decisão receberam o prefixo de implementação: `tbl_imp_revisao` e `tbl_imp_decisao`.

1.4.3. Questões da Seção 1.5 do enunciado

As questões levantadas na especificação original serviram de guia para a delimitação do minimundo. Abaixo, apresentamos as resoluções adotadas pelo grupo com base no recorte do modelo: **Identidade e Autoria**

- **Mudança de instituição após submissão:** No escopo deste minimundo, a relação Pertence entre pesquisador e instituição foi fixada com cardinalidade (0, 1) junto à instituição: cada pesquisador tem no máximo uma instituição. Logo, não há historização de vínculos e os relatórios refletirão a instituição atual associada ao cadastro no momento da consulta.
- **Duas instituições simultâneas:** Pela mesma restrição estrutural (0, 1), o pesquisador registra apenas a sua afiliação principal.
- **Representação da ordem dos autores:** O problema foi resolvido através da alocação do atributo `ordem_autoria` diretamente no relacionamento N:N `É_Autor` (tabela `tbl_pessoa_submissao`), sem alterar a tabela de submissões.
- **Armazenamento de quem adicionou cada autor:** Não é necessário. O escopo de auditoria foi removido. Mantém-se apenas o atributo `ind_responsavel` no relacionamento `É_Autor` para indicar o contato responsável pelo artigo.
- **Prevenção de cadastros duplicados com e-mails diferentes:** O modelo reduz o risco com restrições de unicidade (`UNIQUE`) em `email_principal` e em `cd_orcid`: um pesquisador que informe o ORCID não consegue se cadastrar novamente com outro e-mail. Como o ORCID é opcional, a duplicidade entre cadastros sem ORCID não é impedida pelo banco e fica a cargo da aplicação.
- **Uso do ORCID e sua obrigatoriedade:** O ORCID não é obrigatório, para evitar barreiras de entrada a novos pesquisadores ou a revisores eventuais. Não é utilizado como chave primária, porque uma chave primária não pode ser nula e porque chaves naturais textuais tornam as junções (*JOINS*) mais custosas que chaves artificiais numéricas (*bigint*). Ele foi modelado como um atributo opcional (*nullable*), protegido por uma restrição de unicidade (*UNIQUE*).

Versionamento

- **Um novo upload deve sobrescrever o manuscrito anterior?** No escopo do minimundo simplificado, optou-se por sobrescrever o arquivo (nm_arquivo_pdf direto na tbl_submissao), eliminando o histórico de envios para focar na análise de revisões.
- **Como determinar exatamente qual versão foi disponibilizada aos revisores?** Isso exigiria uma tabela dedicada de versões com chave estrangeira na atribuição. Como o versionamento foi escopado para fora, o sistema assume sempre a versão atual.
- **Uma revisão produzida sobre a versão 2 continua válida após o upload da versão 3?** Numa arquitetura completa, sim, pois a revisão apontaria para o identificador da versão avaliada. No nosso minimundo, como não há múltiplas versões, a situação não ocorre.
- **A versão final é uma nova versão do mesmo objeto ou de outro tipo de artefato?** Logicamente, a versão final difere por ser a versão aceita e formatada, configurando-se como um pós-artefato que, neste recorte, fica fora do escopo de avaliação.
- **Quais metadados precisam ser historizados?** Metadados como histórico de títulos, múltiplos arquivos PDF e registro de mudança de autores. O grupo optou por não historizá-los, mantendo apenas o estado consolidado, adequado às consultas analíticas do Marco 2.

1.5. Modelagem

1.5.1. Modelo conceitual

A Figura 1.1 apresenta o modelo conceitual do banco de dados do ScienceReview, construído no brModelo [brModelo 2026] com a notação de Chen [1976], conforme apresentada por Heuser [2009]. O diagrama reflete o recorte delimitado na Seção 1.4 e usa cardinalidades mínimas e máximas para expressar regras do minimundo, como a obrigatoriedade de ao menos um autor e um tópico por submissão. As Tabelas 1.2 e 1.3 descrevem as entidades e os relacionamentos.

Destacam-se as seguintes características do modelo:

- **Atributos em relacionamentos:** Atributos que pertencem à associação, e não às entidades isoladas, foram alocados nos relacionamentos. Exemplos incluem ordem_autoria em *É_Autor*, nivel_expertise em *Tem_Expertise* e ind_conflito em *É_Revisor*.
- **Entidade associativa:** Para separar a entrega do parecer do convite, o relacionamento *É_Revisor* foi encapsulado como entidade associativa. Isso permite relacionar o convite à tbl_imp_revisao por meio de *Gera_Parecer*, sem armazenar colunas vazias (*NULL*) enquanto a avaliação está pendente.

- **Relacionamentos múltiplos entre as mesmas entidades:** O modelo captura os diferentes papéis de um pesquisador ligando `tbl_pessoa` e `tbl_submissao` por dois relacionamentos paralelos (`É_Autor` e `É_Revisor`), que separam a autoria da avaliação.
- **Autorrelacionamento:** A trilha é representada como um evento filho por meio do relacionamento `Possui_Trilha`, que liga `tbl_evento` a ela mesma com os papéis de evento principal e de trilha.

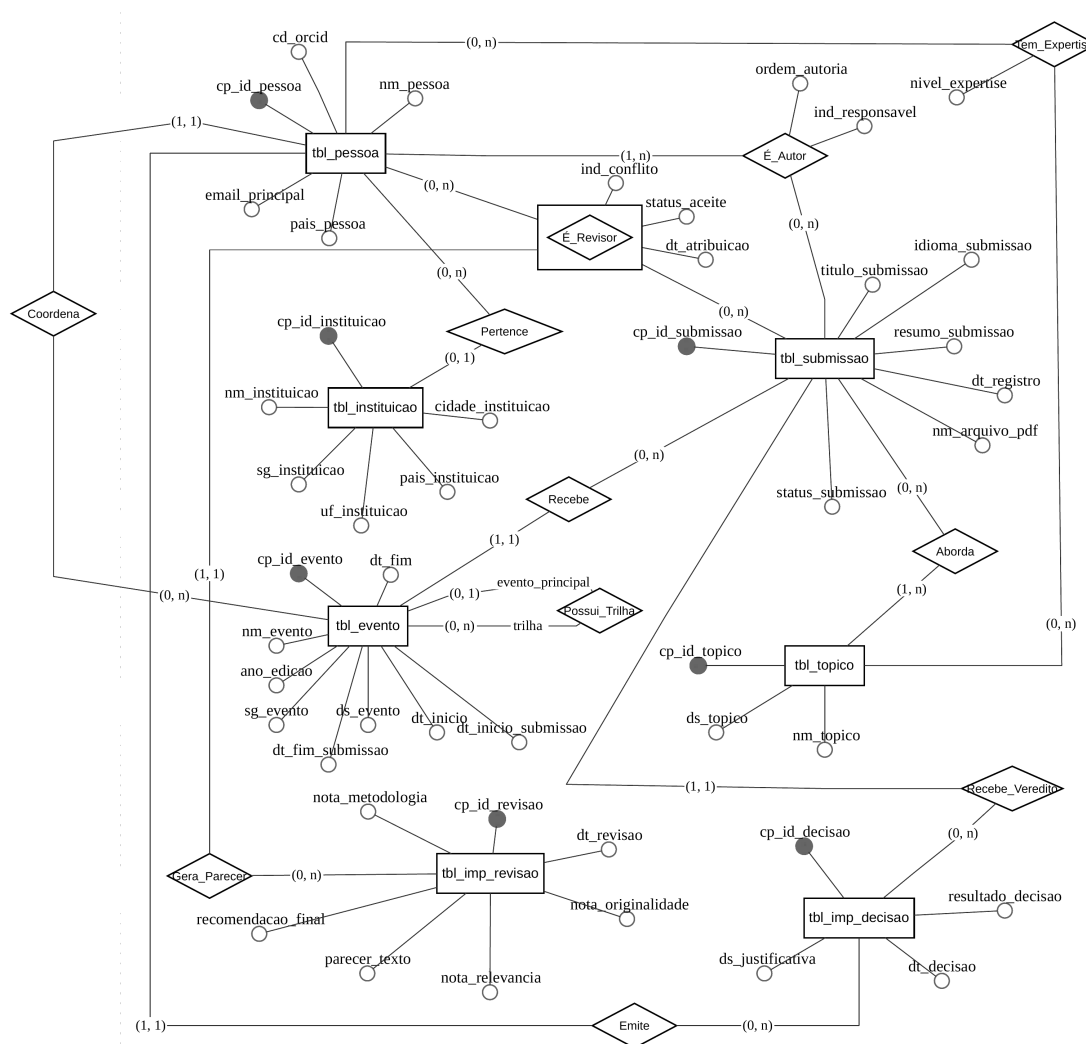


Figura 1.1. Modelo conceitual na notação de Peter Chen (ScienceReview).

1.5.2. Modelo lógico

O modelo lógico (Figura 1.2) traduz o modelo conceitual para o modelo relacional, com 11 relações: as sete entidades e quatro tabelas associativas, uma para cada relacionamento N:N. A Tabela 1.4 lista as colunas de cada relação com seus tipos e chaves; as regras usadas na tradução estão na Seção 1.7.

Tabela 1.2. Entidades do modelo conceitual.

Entidade	Representa	Atributos (chave em negrito)
tbl_pessoa	Pesquisadores e usuários	cp_id_pessoa , nm_pessoa, email_principal, cd_orcid, pais_pessoa
tbl_instituicao	Universidades, centros e empresas	cp_id_instituicao , nm_instituicao, sg_instituicao, pais_instituicao, cidade_instituicao, uf_instituicao
tbl_evento	Evento ou trilha (evento filho)	cp_id_evento , sg_evento, nm_evento, ds_evento, ano_edicao, dt_inicio, dt_fim, dt_inicio_submissao, dt_fim_submissao
tbl_submissao	Trabalho submetido	cp_id_submissao , titulo_submissao, resumo_submissao, idioma_submissao, nm_arquivo_pdf, dt_registro, status_submissao
tbl_topico	Tópico científico	cp_id_topico , nm_topico, ds_topico
tbl_imp_revisao	Parecer de um revisor	cp_id_revisao , dt_revisao, nota_originalidade, nota_metodologia, nota_relevancia, parecer_texto, recomendacao_final
tbl_imp_decisao	Decisão sobre a submissão	cp_id_decisao , resultado_decisao, dt_decisao, ds_justificativa

A figura foi obtida pela conversão automática do brModelo a partir do modelo conceitual, com ajustes manuais de nomes, tipos e chaves. O editor lógico do brModelo oferece apenas tipos genéricos (INT, VARCHAR sem tamanho, FLOAT, DATE, DATETIME e BOOLEAN) e representa toda chave como INT; por isso, a Tabela 1.4 traz os tipos efetivamente usados no PostgreSQL. Duas limitações da ferramenta também afetam a leitura da figura: o autorrelacionamento Possui_Trilha aparece apenas como a coluna ce_evento_pai, sem linha desenhada, e as ligações mostram a cardinalidade (1,1) junto à tabela referenciada mesmo quando a chave estrangeira é opcional, como em ce_instituicao e ce_evento_pai.

Os nomes seguem um padrão de prefixos: tbl_ para tabelas, tbl_imp_ para as tabelas de implementação exigidas pelo enunciado, cp_ para chave primária, ce_ para chave estrangeira, nm_ para nome, dt_ para data, ds_ para descrição, sg_ para sigla e cd_ para código. Atributos de relacionamento mantêm o nome do modelo conceitual (ex.: ordem_autoria, ind_conflito).

Tabela 1.4. Relações do modelo lógico. Chave primária em negrito; chaves estrangeiras indicam a relação referenciada (→); NN = NOT NULL.

Relação	Colunas (tipo e restrições)
tbl_instituicao	cp_id_instituicao (BIGSERIAL); nm_instituicao (VARCHAR(200), NN); pais_instituicao (VARCHAR(100)); sg_instituicao (VARCHAR(100)); cidade_instituicao (VARCHAR(100)); uf_instituicao (VARCHAR(10))

Relação	Colunas (tipo e restrições)
tbl_pessoa	cp_id_pessoa (BIGSERIAL); nm_pessoa (VARCHAR(200), NN); email_principal (VARCHAR(254), NN, único); cd_orcid (VARCHAR(20), único); pais_pessoa (VARCHAR(100)); ce_instituicao (BIGINT → tbl_instituicao)
tbl_evento	cp_id_evento (BIGSERIAL); ce_evento_pai (BIGINT → tbl_evento); sg_evento (VARCHAR(20), NN); nm_evento (VARCHAR(200), NN); ano_edicao (SMALLINT, NN); dt_inicio, dt_fim, dt_inicio_submissao, dt_fim_submissao (DATE, NN); ds_evento (TEXT); ce_coordenador (BIGINT, NN → tbl_pessoa). Único: (sg_evento, ano_edicao)
tbl_submissao	cp_id_submissao (BIGSERIAL); ce_trilha (BIGINT, NN → tbl_evento); titulo_submissao (VARCHAR(500), NN); resumo_submissao (TEXT, NN); idioma_submissao (VARCHAR(20), NN); dt_registro (TIMESTAMP, NN); status_submissao (VARCHAR(20), NN); nm_arquivo_pdf (VARCHAR(200))
tbl_topico	cp_id_topico (BIGSERIAL); nm_topico (VARCHAR(150), NN); ds_topico (TEXT)
tbl_pessoa_submissao (É_Autor)	ce_pessoa (BIGINT → tbl_pessoa); ce_submissao (BIGINT → tbl_submissao); ordem_autoria (INT, NN); ind_responsavel (BOOLEAN, NN). Único: (ce_submissao, ordem_autoria)
tbl_submissao_topico (Aborda)	ce_submissao (BIGINT → tbl_submissao); ce_topico (BIGINT → tbl_topico)
tbl_pessoa_topico (Tem_Expertise)	ce_pessoa (BIGINT → tbl_pessoa); ce_topico (BIGINT → tbl_topico); nivel_expertise (SMALLINT, NN)
tbl_atribuicao_revisao (É_Revisor)	cp_id_atribuicao (BIGSERIAL); ce_pessoa (BIGINT, NN → tbl_pessoa); ce_submissao (BIGINT, NN → tbl_submissao); dt_atribuicao (TIMESTAMP, NN); status_aceite (VARCHAR(20), NN); ind_conflito (BOOLEAN, NN). Único: (ce_pessoa, ce_submissao)
tbl_imp_revisao	cp_id_revisao (BIGSERIAL); nota_metodologia, nota_originalidade, nota_relevancia (NUMERIC(4,2), NN); parecer_texto (TEXT, NN); dt_revisao (TIMESTAMP, NN); recomendacao_final (VARCHAR(30), NN); ce_atribuicao (BIGINT, NN → tbl_atribuicao_revisao)
tbl_imp_decisao	cp_id_decisao (BIGSERIAL); resultado_decisao (VARCHAR(20), NN); ds_justificativa (TEXT); dt_decisao (TIMESTAMP, NN); ce_submissao (BIGINT, NN → tbl_submissao); ce_revisor (BIGINT, NN → tbl_pessoa)

1.6. Rastreabilidade

A Tabela 1.5 relaciona cada entidade e cada relacionamento aos requisitos que ele atende total ou parcialmente, incluindo os requisitos da extensão. Os indicadores do RF42 aparecem entre parênteses.

Tabela 1.3. Relacionamentos do modelo conceitual. A cardinalidade junto a cada entidade indica quantas ocorrências dela se associam a uma ocorrência da outra [Heuser 2009].

Relacionamento	Entidades (cardinalidades)	Tipo	Atributos
Pertence	pessoa (0,n) – instituição (0,1)	1:N	–
Recebe	trilha (1,1) – submissão (0,n)	1:N	–
Possui_Trilha	evento principal (0,1) – trilha (0,n)	1:N (autorrelacionamento)	–
Coordena	pessoa (1,1) – evento (0,n)	1:N	–
É_Autor	pessoa (1,n) – submissão (0,n)	N:N	ordem_autoria, ind_responsavel
Aborda	submissão (0,n) – tópico (1,n)	N:N	–
Tem_Expertise	pessoa (0,n) – tópico (0,n)	N:N	nivel_expertise
É_Revisor	pessoa (0,n) – submissão (0,n)	N:N (entidade associativa)	dt_atribuicao, status_aceite, ind_conflito
Gera_Parecer	É_Revisor (1,1) – revisão (0,n)	1:N	–
Recebe_Veredito	submissão (1,1) – decisão (0,n)	1:N	–
Emitte	pessoa (1,1) – decisão (0,n)	1:N	–

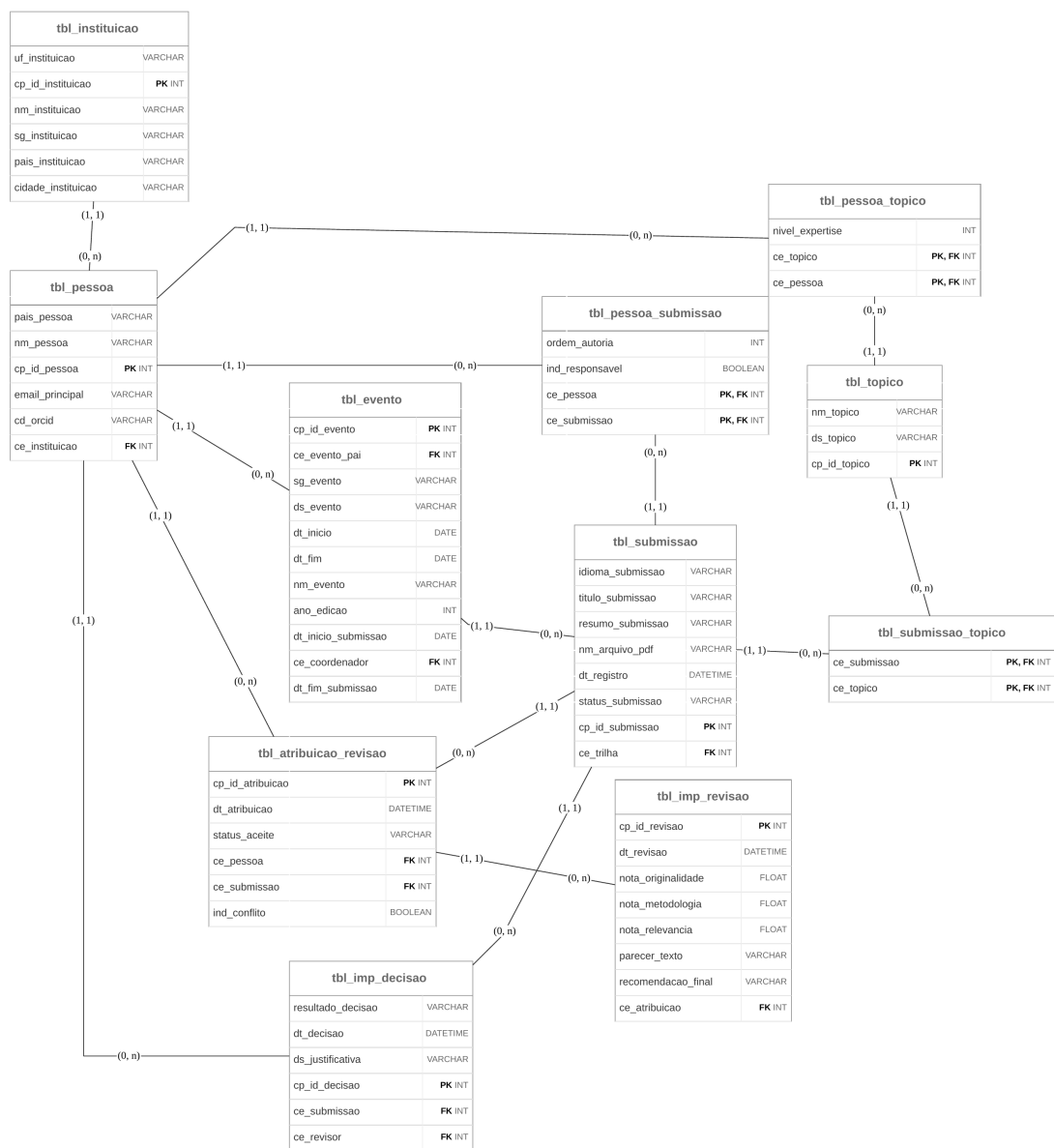


Figura 1.2. Modelo lógico do banco de dados do ScienceReview.

Tabela 1.5. Rastreabilidade entre elementos do modelo e requisitos.

Elemento	Tipo	Requisitos
tbl_pessoa	Entidade	RF3, RF4, RF5, RF7, RF10, RF15, RF31, RF47
tbl_instituicao	Entidade	RF5, RF42 (xv), RF47, RF49
tbl_evento	Entidade	RF1, RF2, RF4, RF6, RF37, RF42 (i, xi), RF48
tbl_submissao	Entidade	RF6, RF7, RF9, RF13, RF42 (i, xiii), RF49
tbl_topico	Entidade	RF9, RF10, RF42 (ii, vii, xii), RF46
tbl_imp_revisao	Entidade	RF25, RF26, RF30, RF42 (ix, x), RF48
tbl_imp_decisao	Entidade	RF31, RF32, RF42 (xi, xii), RF49
Pertence	Relacionamento 1:N	RF5, RF47, RF49
Recebe	Relacionamento 1:N	RF6
Possui_Trilha	Autorrelaciona- mento 1:N	RF1, RF2, RF6
Coordena	Relacionamento 1:N	RF2, RF4, RF40
É_Autor	Relacionamento N:N	RF4, RF7, RF40, RF42 (xv), RF47, RF49
Aborda	Relacionamento N:N	RF9, RF42 (ii, xii), RF46
Tem_Expertise	Relacionamento N:N	RF10, RF42 (vii), RF46
É_Revisor	Entidade associativa	RF4, RF13, RF15, RF17, RF18, RF40, RF42 (iii, viii), RF47, RF48
Gera_Parecer	Relacionamento 1:N	RF25, RF30, RF48
Recebe_Veredito	Relacionamento 1:N	RF31, RF32, RF49
Emite	Relacionamento 1:N	RF31

1.7. Implantação

O banco foi implantado no PostgreSQL [The PostgreSQL Global Development Group 2026] por meio de um único script DDL (`schema.sql`, Anexo A), executado no pgAdmin. O script começa apagando as tabelas existentes (`DROP TABLE IF EXISTS`, em ordem inversa à de criação), o que permite executá-lo novamente sem erros durante os testes. As chaves primárias das entidades e de `tbl_atribuicao_revisao` são artificiais e numéricas (`BIGSERIAL`), geradas automaticamente pelo SGBD; as demais tabelas associativas usam chave primária composta pelas suas chaves estrangeiras. Esta seção descreve a estratégia de tradução do modelo conceitual para o relacional, as restrições de integridade e os índices.

1.7.1. Estratégia de tradução

A tradução seguiu três regras, aplicadas conforme a cardinalidade máxima de cada relacionamento [Heuser 2009]:

1. **Relacionamentos 1:N** viram uma chave estrangeira na tabela do lado N. Quando a cardinalidade mínima junto à entidade referenciada é 1, a coluna recebe `NOT NULL`; quando é 0, a coluna aceita nulo.
2. **Relacionamentos N:N** viram uma tabela associativa, com uma chave estrangeira para cada entidade e chave primária composta pelas duas. Os atributos do relacionamento viram colunas dessa tabela.
3. **A entidade associativa** `É_Revisor` também vira tabela, mas com chave primária artificial (`cp_id_atribuicao`), porque `tbl_imp_revisao` precisa referenciar uma atribuição específica (`Gera_Parecer`). Para não perder a regra do N:N, o par revisor-submissão recebe uma restrição `UNIQUE`.

A Tabela 1.6 mostra como cada relacionamento foi materializado.

O Código 1.1 mostra a tradução de `É_Revisor` e de `Gera_Parecer`, que combina as regras 1 e 3.

1.7.2. Restrições de integridade

Além das chaves, o DDL usa quatro tipos de restrição para materializar regras do mundo:

- `NOT NULL` nos atributos obrigatórios e nas chaves estrangeiras com cardinalidade mínima 1;
- `UNIQUE` em `email_principal` e `cd_orcid`; em (`sg_evento`, `ano_edicao`), que permite repetir a sigla em edições de anos diferentes (RF1); em (`ce_submissao`, `ordem_autoria`), que impede dois autores na mesma posição; e no par revisor-submissão de `É_Revisor`;

Tabela 1.6. Tradução dos relacionamentos do modelo conceitual para o relacional.

Relacionamento	Regra	Materialização
Pertence	1:N	tbl_pessoa.ce_instituicao, aceita nulo (pessoa sem instituição)
Recebe	1:N	tbl_submissao.ce_trilha, NOT NULL
Possui_Trilha	1:N	tbl_evento.ce_evento_pai, autorreferência; nulo no evento principal
Coordena	1:N	tbl_evento.ce_coordenador, NOT NULL
Gera_Parecer	1:N	tbl_imp_revisao.ce_atribuicao, NOT NULL
Recebe_Veredito	1:N	tbl_imp_decisao.ce_submissao, NOT NULL
Emite	1:N	tbl_imp_decisao.ce_revisor, NOT NULL
É_Autor	N:N	tbl_pessoa_submissao, PK (ce_pessoa, ce_submissao)
Aborda	N:N	tbl_submissao_topico, PK (ce_submissao, ce_topico)
Tem_Expertise	N:N	tbl_pessoa_topico, PK (ce_pessoa, ce_topico)
É_Revisor	Entidade associativa	tbl_atribuicao_revisao, PK cp_id_atribuicao e UNIQUE (ce_pessoa, ce_submissao)

Código 1.1. Tradução da entidade associativa É_Revisor e do relacionamento Gera_Parecer (trecho do schema.sql).

```

CREATE TABLE tbl_atribuicao_revisao (
    cp_id_atribuicao BIGSERIAL PRIMARY KEY,
    ce_pessoa BIGINT NOT NULL,
    ce_submissao BIGINT NOT NULL,
    dt_atribuicao TIMESTAMP NOT NULL,
    status_aceite VARCHAR(20) NOT NULL DEFAULT 'pendente'
    CHECK (status_aceite IN ('pendente', 'aceito', 'recusado', 'removido')),
    ind_conflito BOOLEAN NOT NULL DEFAULT FALSE,
    UNIQUE (ce_pessoa, ce_submissao),
    FOREIGN KEY (ce_pessoa) REFERENCES tbl_pessoa (cp_id_pessoa),
    FOREIGN KEY (ce_submissao) REFERENCES tbl_submissao (cp_id_submissao)
);

CREATE TABLE tbl_imp_revisao (
    cp_id_revisao BIGSERIAL PRIMARY KEY,
    ...
    ce_atribuicao BIGINT NOT NULL,
    FOREIGN KEY (ce_atribuicao) REFERENCES tbl_atribuicao_revisao (cp_id_atribuicao)
);

```

- CHECK nos domínios: situação da submissão, do convite, da recomendação e da decisão restritas a listas de valores; notas entre 0 e 10; nível de expertise entre 1 e 5; ordem de autoria a partir de 1; e datas de término posteriores às de início;
- DEFAULT para valores iniciais: submissão 'submetida', convite 'pendente' e conflito FALSE.

Algumas regras envolvem mais de uma linha ou tabela e não podem ser expressas com essas restrições simples: ao menos um autor e um tópico por submissão, ao menos um autor responsável, trilha que não tenha trilhas próprias e submissão apontando sempre para uma trilha, e não para o evento principal. Essas regras ficam a cargo da aplicação e serão revistas no Marco 2.

1.7.3. Índices

O PostgreSQL cria índices automaticamente para as chaves primárias e para as restrições UNIQUE, mas não para as chaves estrangeiras [The PostgreSQL Global Development Group 2026]. Como as consultas previstas para os indicadores (RF42 e RF46 a RF49) fazem junções por essas colunas, o grupo criou um índice para cada chave estrangeira usada nas junções que ainda não era a primeira coluna de uma chave primária ou única (Código 1.2). Por exemplo, a chave primária (ce_submissao, ce_topico) de tbl_submissao_topico já acelera buscas por submissão, mas não por tópico. Essa foi a otimização do modelo físico adotada neste marco; a análise de planos de execução fica para o Marco 2.

Código 1.2. Índices criados nas chaves estrangeiras (trecho do schema.sql).

```
CREATE INDEX idx_pessoa_instituicao ON tbl_pessoa (ce_instituicao);
CREATE INDEX idx_submissao_trilha ON tbl_submissao (ce_trilha);
CREATE INDEX idx_submissao_topico_topico ON tbl_submissao_topico (ce_topico);
CREATE INDEX idx_pessoa_topico_topico ON tbl_pessoa_topico (ce_topico);
CREATE INDEX idx_atribuicao_submissao ON tbl_atribuicao_revisao (ce_submissao);
CREATE INDEX idx_revisao_atribuicao ON tbl_imp_revisao (ce_atribuicao);
CREATE INDEX idx_decisao_submissao ON tbl_imp_decisao (ce_submissao);
```

1.8. População e exploração de metadados

1.8.1. População

O banco foi populado com dados sintéticos por meio de um script de INSERT por tabela, executados no pgAdmin após o DDL, na ordem do Anexo B. Cada script tem um único comando INSERT INTO ... VALUES, com um registro por linha. As 1.000 instituições usam nomes de universidades reais, e as 3.500 pessoas são fictícias, cada uma vinculada a uma instituição.

Os dados das demais tabelas foram gerados com consultas SQL no próprio PostgreSQL, seguindo regras fixas que mantêm o ciclo de avaliação coerente, e depois gravados como INSERT com valores literais:

- três eventos (SBBD, SBES e SBRC) em duas edições (2025 e 2026), cada um com duas trilhas, totalizando 18 linhas em `tbl_evento`;
- 20 tópicos; cada pessoa declara expertise em um ou dois tópicos, com nível de 1 a 5;
- 1.700 submissões distribuídas entre as 12 trilhas, registradas dentro do período de submissão da trilha, cada uma com três autores (o primeiro é o responsável) e um ou dois tópicos;
- três revisores por submissão, nunca autores da própria submissão; o convite pode estar aceito, recusado, pendente ou removido, e a atribuição é marcada com `ind_conflito` quando o revisor é da mesma instituição de algum autor (RF47);
- um parecer para cada convite aceito, entregue entre 5 e 29 dias após a atribuição (RF48), com notas maiores nas submissões aceitas;
- uma decisão final por submissão, emitida pelo coordenador da trilha; em 10% das submissões, uma decisão anterior de revisão solicitada é preservada (RF32).

A Tabela 1.7 mostra o volume final. Três tabelas passam de 5.000 registros: `tbl_pessoa_topico`, `tbl_pessoa_submissao` e `tbl_atribuicao_revisao`.

Tabela 1.7. Volume e espaço em disco de cada tabela após a população.

Tabela	Registros	Dados	Índices	Total
<code>tbl_pessoa_topico</code>	5.250	272 kB	240 kB	536 kB
<code>tbl_atribuicao_revisao</code>	5.100	368 kB	496 kB	888 kB
<code>tbl_pessoa_submissao</code>	5.100	264 kB	408 kB	696 kB
<code>tbl_pessoa</code>	3.500	328 kB	496 kB	856 kB
<code>tbl_submissao_topico</code>	2.550	112 kB	136 kB	272 kB
<code>tbl_imp_revisao</code>	2.550	376 kB	144 kB	552 kB
<code>tbl_imp_decisao</code>	1.870	200 kB	136 kB	368 kB
<code>tbl_submissao</code>	1.700	280 kB	96 kB	408 kB
<code>tbl_instituicao</code>	1.000	96 kB	40 kB	168 kB
<code>tbl_topico</code>	20	8 kB	16 kB	32 kB
<code>tbl_evento</code>	18	8 kB	32 kB	48 kB
Total	28.658			

Após a carga, o grupo conferiu com consultas simples que os dados respeitam as regras que o banco não garante sozinho: toda submissão tem autor responsável e ao menos um tópico, nenhuma submissão aponta para um evento principal, nenhum revisor é autor da submissão que revisa e a situação de cada submissão corresponde à sua decisão mais recente.

1.8.2. Exploração de metadados

O PostgreSQL guarda a descrição do próprio banco em catálogos que podem ser consultados com SQL: o `information_schema`, definido pelo padrão SQL, e o `pg_catalog`,

próprio do PostgreSQL [The PostgreSQL Global Development Group 2026]. O script metadados.sql (Anexo B) reúne sete consultas sobre esses catálogos, resumidas na Tabela 1.8.

Tabela 1.8. Consultas de exploração de metadados e seus resultados.

#	O que mostra	Catálogo	Resultado
1	Tabelas do esquema	information_ schema.tables	11 tabelas
2	Colunas, tipos e nulidade	information_ schema.columns	63 colunas
3	Chaves primárias, únicas e estrangeiras	table_constraints, key_ column_usage	11 PK, 5 UNIQUE, 15 FK
4	Destino de cada chave estrangeira	constraint_column_ usage	15 referências
5	Restrições CHECK	check_constraints	11 restrições
6	Índices	pg_indexes	23 índices (16 automáticos e 7 criados)
7	Registros e espaço em disco	pg_stat_user_tables	Tabela 1.7

O Código 1.3 mostra a consulta 4, que reconstrói a partir do catálogo as ligações do modelo lógico: cada uma das 15 chaves estrangeiras aparece com a tabela e a coluna de origem e de destino, o que permitiu conferir que o DDL implementa todos os relacionamentos 1:N e N:N da Tabela 1.6.

Código 1.3. Consulta ao catálogo: destino de cada chave estrangeira.

```
SELECT kcu.table_name AS tabela_origem,
       kcu.column_name AS coluna_origem,
       ccu.table_name AS tabela_destino,
       ccu.column_name AS coluna_destino
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu
  ON kcu.constraint_name = tc.constraint_name
 AND kcu.table_schema = tc.table_schema
JOIN information_schema.constraint_column_usage ccu
  ON ccu.constraint_name = tc.constraint_name
 AND ccu.table_schema = tc.table_schema
WHERE tc.table_schema = 'public'
 AND tc.constraint_type = 'FOREIGN KEY'
ORDER BY tabela_origem, coluna_origem;
```

A exploração dos metadados trouxe duas observações:

- **Restrições reescritas pelo SGBD:** o catálogo não guarda o CHECK exatamente como foi escrito. Por exemplo, nivel_expertise BETWEEN 1 AND 5 aparece como (nivel_expertise >= 1) AND (nivel_expertise <= 5), e as listas IN (...) aparecem como comparações com um vetor (= ANY (ARRAY[...])). O PostgreSQL também dá nomes automáticos às restrições sem nome (ex.: tbl_evento_sg_evento_ano_edicao_key).

- **Custo dos índices:** em várias tabelas os índices ocupam mais espaço que os próprios dados (em `tbl_atribuicao_revisao`, 496 kB contra 368 kB), porque elas têm poucas colunas e várias chaves. É o custo de espaço pago para acelerar as junções, e será avaliado junto com o desempenho das consultas no Marco 2.

Referências

- brModelo (2026). brModeloWeb: ferramenta de modelagem de banco de dados. <https://www.brmodeloweb.com/>. Acesso em: out. 2026.
- Chen, P. P.-S. (1976). The entity-relationship model: Toward a unified view of data. *ACM Transactions on Database Systems*, 1(1):9–36.
- Heuser, C. A. (2009). *Projeto de Banco de Dados*, volume 4 of *Série Livros Didáticos Informática UFRGS*. Bookman, Porto Alegre, 6 edition.
- The PostgreSQL Global Development Group (2026). PostgreSQL documentation. <https://www.postgresql.org/docs/>. Acesso em: out. 2026.
- Universidade Federal da Bahia (2026). CASO 1 – MATA60: ScienceReview. Instituto de Computação, Departamento de Ciência da Computação. Enunciado da disciplina MATA60 – Bancos de Dados.

Anexo A – Script DDL

Conteúdo completo do arquivo `schema.sql`.

Código 1.4. Script DDL completo (`schema.sql`).

```
/* A chave primária deve autoincrementar via BIGSERIAL */

/*
ESTRATÉGIA DE TRADUÇÃO DO MODELO CONCEITUAL (PDF) PARA O RELACIONAL:
- Relacionamento 1:1 ou 1:N -> vira FK direto numa das tabelas existentes (lado "N" recebe a FK).
- Relacionamento N:M (ambos os lados podem ter mais de uma ocorrência) -> vira uma tabela própria
  (tabela associativa), com uma FK pra cada entidade participante + os atributos que são do
  PRÓPRIO relacionamento (não pertencem a nenhuma das duas entidades isoladamente).
Tabelas associativas deste arquivo: tbl_pessoa_submissao (É_Autor), tbl_submissao_topico (Aborda),
tbl_pessoa_topico (Tem_Expertise) e tbl_atribuicao_revisao (É_Revisor).
*/

/* Apaga as tabelas (se existirem) para o script poder ser executado de novo no pgAdmin.
A ordem é a inversa da criação: primeiro quem tem FK, depois quem é referenciado. */
DROP TABLE IF EXISTS tbl_imp_decisao;
DROP TABLE IF EXISTS tbl_imp_revisao;
DROP TABLE IF EXISTS tbl_atribuicao_revisao;
DROP TABLE IF EXISTS tbl_pessoa_topico;
DROP TABLE IF EXISTS tbl_submissao_topico;
DROP TABLE IF EXISTS tbl_topico;
DROP TABLE IF EXISTS tbl_pessoa_submissao;
DROP TABLE IF EXISTS tbl_submissao;
DROP TABLE IF EXISTS tbl_evento;
DROP TABLE IF EXISTS tbl_pessoa;
DROP TABLE IF EXISTS tbl_instituicao;

CREATE TABLE tbl_instituicao (
    cp_id_instituicao BIGSERIAL PRIMARY KEY,
```

```

nm_instituicao VARCHAR(200) NOT NULL,
pais_instituicao VARCHAR(100), /* nome do país por extenso (ex.: 'Brasil') */
sg_instituicao VARCHAR(100),
cidade_instituicao VARCHAR(100),
uf_instituicao VARCHAR(10) /* UF ou província; NULL para instituições fora do Brasil que não
    usam UF */
);

CREATE TABLE tbl_pessoa (
    cp_id_pessoa BIGSERIAL PRIMARY KEY,
    nm_pessoa VARCHAR(200) NOT NULL,
    email_principal VARCHAR(254) NOT NULL UNIQUE,
    cd_orcid VARCHAR(20) UNIQUE, /* opcional (aceita NULL), mas não pode repetir */
    pais_pessoa VARCHAR(100),
    ce_instituicao BIGINT, /* FK do relacionamento 'Pertence': cardinalidade (0,1) do lado da pessoa
        , por isso aceita NULL */
    FOREIGN KEY (ce_instituicao) REFERENCES tbl_instituicao (cp_id_instituicao)
);

CREATE TABLE tbl_evento (
    cp_id_evento BIGSERIAL PRIMARY KEY,
    ce_evento_pai BIGINT, /* Relacionamento recursivo: trilha é tratada como um "sub-evento", ou
        seja, uma linha de tbl_evento cujo ce_evento_pai aponta para o evento principal. NULL = evento
        raiz; preenchido = linha que representa uma trilha */
    sg_evento VARCHAR(20) NOT NULL,
    nm_evento VARCHAR(200) NOT NULL,
    ano_edicao SMALLINT NOT NULL,
    dt_inicio DATE NOT NULL,
    dt_fim DATE NOT NULL,
    ds_evento TEXT,
    dt_fim_submissao DATE NOT NULL,
    dt_inicio_submissao DATE NOT NULL,
    ce_coordenador BIGINT NOT NULL, /* FK do relacionamento 'Coordena': todo evento e toda trilha
        tem uma pessoa coordenadora */
    UNIQUE (sg_evento, ano_edicao), /* a mesma sigla pode se repetir em anos diferentes (edições),
        mas não no mesmo ano */
    CHECK (dt_fim >= dt_inicio),
    CHECK (dt_fim_submissao >= dt_inicio_submissao),
    FOREIGN KEY (ce_coordenador) REFERENCES tbl_pessoa (cp_id_pessoa),
    FOREIGN KEY (ce_evento_pai) REFERENCES tbl_evento (cp_id_evento)
);

CREATE TABLE tbl_submissao (
    cp_id_submissao BIGSERIAL PRIMARY KEY,
    ce_trilha BIGINT NOT NULL,
    titulo_submissao VARCHAR(500) NOT NULL,
    resumo_submissao TEXT NOT NULL,
    idioma_submissao VARCHAR(20) NOT NULL,
    dt_registro TIMESTAMP NOT NULL,
    status_submissao VARCHAR(20) NOT NULL DEFAULT 'submetida'
    CHECK (status_submissao IN ('submetida', 'em_revisao', 'aceita', 'rejeitada')),
    nm_arquivo_pdf VARCHAR(200), /* nome do arquivo PDF do manuscrito (atributo nm_arquivo_pdf do
        modelo conceitual) */
    FOREIGN KEY (ce_trilha) REFERENCES tbl_evento (cp_id_evento) /* ce_trilha referencia tbl_evento
        pq a trilha é uma linha de tbl_evento (ver comentário em ce_evento_pai) */
);

/* TABELA DE RELACIONAMENTO 'É Autor' ENTRE PESSOA E SUBMISSÃO, POIS UMA PESSOA PODE TER VÁRIAS
    SUBMISSÕES E UMA SUBMISSÃO PODE TER VÁRIOS AUTORES */
CREATE TABLE tbl_pessoa_submissao (
    ce_pessoa BIGINT NOT NULL,
    ce_submissao BIGINT NOT NULL,
    ordem_autoria INT NOT NULL CHECK (ordem_autoria >= 1), /* posição do autor na lista de autoria
        (1º, 2º, 3º autor...) */
    is_responsavel BOOLEAN NOT NULL, /* marca o(s) autor(es) responsável(is) pela submissão (quem
        gerencia o envio, recebe notificações etc. - RF7). Atenção: o banco não garante sozinho que
        pelo menos um autor por submissão tenha TRUE; isso precisa ser checado na aplicação */
    PRIMARY KEY (ce_pessoa, ce_submissao),
    UNIQUE (ce_submissao, ordem_autoria), /* dois autores da mesma submissão não podem ocupar a
        mesma posição */

```

```

        FOREIGN KEY (ce_pessoa) REFERENCES tbl_pessoa (cp_id_pessoa),
        FOREIGN KEY (ce_submissao) REFERENCES tbl_submissao (cp_id_submissao)
    );

CREATE TABLE tbl_topico (
    cp_id_topico BIGSERIAL PRIMARY KEY,
    nm_topico VARCHAR(150) NOT NULL,
    ds_topico TEXT
);

/* TABELA DE RELACIONAMENTO 'Aborda' ENTRE SUBMISSÃO E TÓPICO, POIS UMA SUBMISSÃO PODE TER VÁRIOS TÓPICOS E UM TÓPICO PODE TER VÁRIAS SUBMISSÕES */
CREATE TABLE tbl_submissao_topico (
    ce_submissao BIGINT NOT NULL,
    ce_topico BIGINT NOT NULL,
    PRIMARY KEY (ce_submissao, ce_topico),
    FOREIGN KEY (ce_submissao) REFERENCES tbl_submissao (cp_id_submissao),
    FOREIGN KEY (ce_topico) REFERENCES tbl_topico (cp_id_topico)
);

/* TABELA DE RELACIONAMENTO 'Tem_Expertise' ENTRE PESSOA E TÓPICO, POIS UMA PESSOA PODE TER VÁRIOS TÓPICOS E UM TÓPICO PODE TER VÁRIAS PESSOAS */
CREATE TABLE tbl_pessoa_topico (
    ce_pessoa BIGINT NOT NULL,
    ce_topico BIGINT NOT NULL,
    nivel_expertise SMALLINT NOT NULL CHECK (nivel_expertise BETWEEN 1 AND 5), /* escala de 1 (baixo) a 5 (alto); numérica para servir de peso na aderência temática (RF46) */
    PRIMARY KEY (ce_pessoa, ce_topico),
    FOREIGN KEY (ce_pessoa) REFERENCES tbl_pessoa (cp_id_pessoa),
    FOREIGN KEY (ce_topico) REFERENCES tbl_topico (cp_id_topico)
);

/*TABELA DE RELACIONAMENTO 'É_Revisor' ENTRE PESSOA E SUBMISSÃO, POIS UMA PESSOA PODE SER REVISOR DE VÁRIAS SUBMISSÕES E UMA SUBMISSÃO PODE TER VÁRIOS REVISORES */
CREATE TABLE tbl_atribuicao_revisao (
    cp_id_atribuicao BIGSERIAL PRIMARY KEY, /* PK substituta (em vez de chave composta ce_pessoa+ ce_submissao) pois tbl_imp_revisao precisa referenciar uma atribuição específica ( relacionamento Gera_Parecer) */
    ce_pessoa BIGINT NOT NULL,
    ce_submissao BIGINT NOT NULL,
    dt_atribuicao TIMESTAMP NOT NULL,
    status_aceite VARCHAR(20) NOT NULL DEFAULT 'pendente'
        CHECK (status_aceite IN ('pendente', 'aceito', 'recusado', 'removido')),
    flag_conflito BOOLEAN NOT NULL DEFAULT FALSE,
    UNIQUE (ce_pessoa, ce_submissao), /* mesmo com a PK substituta, o par revisor+submissão continua único (relacionamento N:N) */
    FOREIGN KEY (ce_pessoa) REFERENCES tbl_pessoa (cp_id_pessoa),
    FOREIGN KEY (ce_submissao) REFERENCES tbl_submissao (cp_id_submissao)
);

CREATE TABLE tbl_imp_revisao (
    cp_id_revisao BIGSERIAL PRIMARY KEY,
    nota_metodologia NUMERIC(4,2) NOT NULL CHECK (nota_metodologia BETWEEN 0 AND 10),
    nota_originalidade NUMERIC(4,2) NOT NULL CHECK (nota_originalidade BETWEEN 0 AND 10),
    nota_relevancia NUMERIC(4,2) NOT NULL CHECK (nota_relevancia BETWEEN 0 AND 10),
    parecer_texto TEXT NOT NULL,
    dt_revisao TIMESTAMP NOT NULL,
    recomendacao_final VARCHAR(30) NOT NULL
        CHECK (recomendacao_final IN ('aceitar', 'aceitar_com_ressalvas', 'rejeitar')),
    ce_atribuicao BIGINT NOT NULL, /* FK do relacionamento 'Gera_Parecer': o parecer vem de uma atribuição específica (não referencia pessoa/submissão direto, pra não duplicar o que já está em tbl_atribuicao_revisao) */
    FOREIGN KEY (ce_atribuicao) REFERENCES tbl_atribuicao_revisao (cp_id_atribuicao)
);

/* Materializa dois relacionamentos do modelo conceitual: Recebe_Veredito (submissão -> decisão, 1:N , pois decisões anteriores são preservadas - RF32) e Emite (pessoa -> decisão, 1:N) */
CREATE TABLE tbl_imp_decisao (
    cp_id_decisao BIGSERIAL PRIMARY KEY,

```

```

    resultado_decisao VARCHAR(20) NOT NULL
        CHECK (resultado_decisao IN ('aceita', 'rejeitada', 'revisao_solicitada')),
    ds_justificativa TEXT,
    dt_decisao TIMESTAMP NOT NULL, /* obrigatória: a decisão vigente é a de data mais recente (RF32)
    */
    ce_submissao BIGINT NOT NULL, /* FK do relacionamento Recebe_Veredito */
    ce_revisor BIGINT NOT NULL, /* FK do relacionamento Emite: pessoa que emite/decide o veredito */
    FOREIGN KEY (ce_submissao) REFERENCES tbl_submissao (cp_id_submissao),
    FOREIGN KEY (ce_revisor) REFERENCES tbl_pessoa (cp_id_pessoa)
);

/*
ÍNDICES (modelo físico):
O PostgreSQL cria índice automaticamente para PRIMARY KEY e UNIQUE, mas NÃO para FOREIGN KEY.
Criamos índices nas FKs usadas nas junções das consultas (RF42, RF46-RF49) que ainda não estão
cobertas como primeira coluna de uma PK ou UNIQUE. Ex.: em tbl_submissao_topico a PK (ce_submissao,
ce_topico)
já acelera buscas por ce_submissao, mas não por ce_topico.
*/
CREATE INDEX idx_pessoa_instituicao ON tbl_pessoa (ce_instituicao);
CREATE INDEX idx_submissao_trilha ON tbl_submissao (ce_trilha);
CREATE INDEX idx_submissao_topico_topico ON tbl_submissao_topico (ce_topico);
CREATE INDEX idx_pessoa_topico_topico ON tbl_pessoa_topico (ce_topico);
CREATE INDEX idx_atribuicao_submissao ON tbl_atribuicao_revisao (ce_submissao);
CREATE INDEX idx_revisao_atribuicao ON tbl_imp_revisao (ce_atribuicao);
CREATE INDEX idx_decisao_submissao ON tbl_imp_decisao (ce_submissao);

```

Anexo B – Roteiro de reprodução

A reprodução usa apenas o PostgreSQL e o pgAdmin. Os arquivos estão listados no Anexo C.

1. No pgAdmin, criar um banco de dados vazio: botão direito em *Databases* → *Create* → *Database...*, com o nome *sciencereview*.
2. Selecionar o banco criado e abrir o *Query Tool* (menu *Tools* → *Query Tool*).
3. Abrir cada arquivo abaixo pelo ícone *Open File* e executá-lo com F5, sem nenhum trecho selecionado, nesta ordem:
 - (a) *schema.sql*;
 - (b) *populate_instituicao.sql*;
 - (c) *populate_pessoa.sql*;
 - (d) *populate_evento.sql*;
 - (e) *populate_topico.sql*;
 - (f) *populate_pessoa_topico.sql*;
 - (g) *populate_submissao.sql*;
 - (h) *populate_pessoa_submissao.sql*;
 - (i) *populate_submissao_topico.sql*;
 - (j) *populate_atribuicao_revisao.sql*;
 - (k) *populate_imp_revisao.sql*;

(l) populate_imp_decisao.sql.

4. Abrir o metadados.sql (Código 1.5), executar a linha ANALYZE; e depois selecionar uma consulta por vez e executá-la com F5. O *Query Tool* mostra apenas o resultado do último SELECT executado.

Os scripts de população dependem de os identificadores começarem em 1. Se algum arquivo falhar no meio da sequência, basta recomeçar pelo schema.sql, que apaga e recria todas as tabelas.

Código 1.5. Consultas de exploração de metadados (metadados.sql).

```
/*
EXPLORAÇÃO DE METADADOS

Executar depois do schema.sql e de todos os populates.

Como rodar no pgAdmin: o Query Tool mostra só o resultado do ÚLTIMO SELECT do script.
Por isso, selecione UMA consulta por vez (do SELECT até o ponto e vírgula) e aperte F5.

- information_schema: catálogo padrão do SQL (ISO), igual em vários SGBDs.
- pg_catalog / pg_stat_*: catálogo próprio do PostgreSQL (tamanho em disco, índices, estatísticas).
*/

/* Atualiza as estatísticas das tabelas, para as contagens da consulta 7 ficarem exatas */
ANALYZE;

/* -----
1) Tabelas criadas no esquema public
----- */
SELECT table_name, table_type
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;

/* -----
2) Colunas de cada tabela: tipo, tamanho, se aceita NULL e valor padrão
----- */
SELECT table_name,
       column_name,
       data_type,
       character_maximum_length AS tamanho_texto,
       numeric_precision AS precisao,
       numeric_scale AS casas_decimais,
       is_nullable AS aceita_nulo,
       column_default AS valor_padrao
FROM information_schema.columns
WHERE table_schema = 'public'
ORDER BY table_name, ordinal_position;

/* -----
3) Chaves primárias, únicas e estrangeiras, com as colunas envolvidas
----- */
SELECT tc.table_name,
       tc.constraint_type,
       tc.constraint_name,
       kcu.column_name
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu
  ON kcu.constraint_name = tc.constraint_name
 AND kcu.table_schema = tc.table_schema
WHERE tc.table_schema = 'public'
ORDER BY tc.table_name, tc.constraint_type, tc.constraint_name, kcu.ordinal_position;

/* -----
4) Para onde cada chave estrangeira aponta (tabela e coluna de origem -> destino)
----- */
```

```

SELECT kcu.table_name AS tabela_origem,
       kcu.column_name AS coluna_origem,
       ccu.table_name AS tabela_destino,
       ccu.column_name AS coluna_destino
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu
      ON kcu.constraint_name = tc.constraint_name
      AND kcu.table_schema = tc.table_schema
JOIN information_schema.constraint_column_usage ccu
      ON ccu.constraint_name = tc.constraint_name
      AND ccu.table_schema = tc.table_schema
WHERE tc.table_schema = 'public'
      AND tc.constraint_type = 'FOREIGN KEY'
ORDER BY tabela_origem, coluna_origem;

/* -----
5) Restrições CHECK (regras de domínio) e a condição de cada uma.
   O PostgreSQL também lista os NOT NULL como CHECK; eles são filtrados aqui.
----- */
SELECT tc.table_name,
       cc.constraint_name,
       cc.check_clause
FROM information_schema.check_constraints cc
JOIN information_schema.table_constraints tc
      ON tc.constraint_name = cc.constraint_name
      AND tc.constraint_schema = cc.constraint_schema
WHERE cc.constraint_schema = 'public'
      AND cc.constraint_name NOT LIKE '%_not_null'
ORDER BY tc.table_name, cc.constraint_name;

/* -----
6) Índices: os criados automaticamente (PK e UNIQUE, terminam em _pkey e _key)
   e os criados no schema.sql para as chaves estrangeiras (começam com idx_)
----- */
SELECT tablename AS tabela,
       indexname AS indice,
       indexdef AS definicao
FROM pg_indexes
WHERE schemaname = 'public'
ORDER BY tablename, indexname;

/* -----
7) Quantidade de linhas e espaço em disco de cada tabela
   (dados, índices e total), da maior para a menor
----- */
SELECT relname AS tabela,
       n_live_tup AS linhas,
       pg_size_pretty(pg_relation_size(relid)) AS tamanho_dados,
       pg_size_pretty(pg_indexes_size(relid)) AS tamanho_indices,
       pg_size_pretty(pg_total_relation_size(relid)) AS tamanho_total
FROM pg_stat_user_tables
ORDER BY n_live_tup DESC;

```

Anexo C – Arquivos enviados com o relatório

Os arquivos abaixo foram enviados junto com este relatório, como documentos auxiliares para a reprodução do projeto (Anexo B):

- `schema.sql`: script DDL (Anexo A);
- `populate_*.sql`: 11 scripts de população, um por tabela, executados na ordem do Anexo B;
- `metadados.sql`: consultas de exploração de metadados (Código 1.5);

- `modelo_conceitual.pdf` e `modelo_logico.pdf`: modelos conceitual e lógico exportados do brModelo, em versão vetorial das Figuras 1.1 e 1.2.